

PROJETO DE DEEP LEARNING

PlantVillage: classificação multiclasse

Escolhemos uma base conhecida para aplicar CNN, transfer learning, avaliação e reprodutibilidade em um projeto completo.

TensorFlow

PlantVillage

MobileNetV2

38 classes

Pipeline do experimento

PlantVillage54.303 imagens
38 classes**95,40%**

acurácia no teste

**tf.data**split 70/15/15
resize 224x224**94,41%**

F1 macro



TensorFlow + MobileNetV2

MobileNetV2transfer learning
softmax 38 classes**8.146**

imagens de teste

O PlantVillage já é uma base conhecida na área. Isso permitiu focar no que era mais importante para o trabalho: **construir e avaliar o modelo.**

Base conhecida

Disponível no TensorFlow Datasets, com classes já organizadas.

Escopo adequado

Volume suficiente para treino, validação e teste.

Foco do trabalho

Aplicar o conteúdo estudado em deep learning.

Classificador reprodutível para 38 classes

O projeto cobre o fluxo inteiro: dados, modelo, treino, avaliação, predição e artefatos versionados.

1

Pacote Python com comandos de treino, avaliação e predição.

2

Treinar uma CNN baseline e um modelo com transfer learning.

3

Modelo, métricas, gráficos e metadados salvos no repositório.

Dados padronizados para um problema de classificação multiclasse

54.303

imagens no experimento completo

38

classes de planta, doença e folha saudável

224

pixels por lado após redimensionamento

Parte	Proporção	Quantidade	Uso
Treino	70%	38.012	Aprender os pesos
Validação	15%	8.145	Acompanhar generalização
Teste	15%	8.146	Medir resultado final

● Split calculado no código: 38.012 treino, 8.145 validação e 8.146 teste.

Pipeline de dados

O carregamento ficou em `data.py`. Mantivemos o experimento determinístico e com buffers limitados para não consumir CPU e RAM sem necessidade.

```
bundle = load_datasets(config)
train_ds = bundle.train
val_ds = bundle.val
```

A

`tfds.load(..., shuffle\_files=False)`.

B

Shuffle determinístico antes do split 70/15/15.

C

Resize `224x224`, cast para `float32`, batch e prefetch.

D

Data augmentation apenas durante o treino.

Arquitetura

- Entrada `224x224x3`.
- Augmentation: flip horizontal, rotação 0.08 e zoom 0.10.
- Pré-processamento MobileNetV2: escala `[-1, 1]`.
- Backbone MobileNetV2 `include\_top=False`, pesos ImageNet.
- Topo: GlobalAveragePooling2D, Dropout 0.3 e Dense softmax.

```
model = build_model(
    "mobilenet",
    num_classes=38,
    config=config,
    fine_tune=False,
)
```

Componente	Função
Data augmentation	Regularização no treino
MobileNetV2	Backbone convolucional pré-treinado
GlobalAveragePooling2D	Reduzir mapas de características
Dropout	Regularização do classificador
Dense softmax	Gerar probabilidade para 38 classes

● O backbone ficou congelado no treino principal; o projeto suporta fine tuning por flag.

Treino em GPU com limite de recursos

20

épocas

8

batch size

2

CPUs no Docker

8 GB

limite de RAM

Hiperparâmetros e callbacks

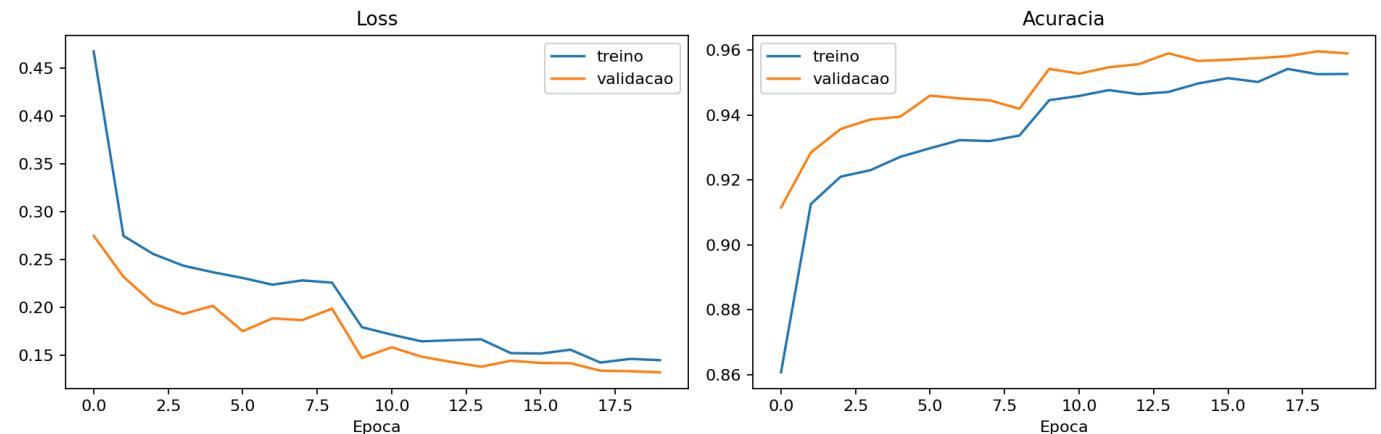
- Adam com learning rate inicial ``1e-3``.
- Loss ``sparse_categorical_crossentropy``.
- Checkpoint pelo melhor ``val_accuracy``.
- EarlyStopping monitorando ``val_loss``.
- ReduceLROnPlateau quando ``val_loss`` estabiliza.

GPU reduziu a primeira época de aproximadamente 944s para 111s.

Curva de treino

A validação sobe rápido nas primeiras épocas. Depois o ganho fica menor e o learning rate cai automaticamente.

Melhor `val\_accuracy`: 0.9596. Menor `val\_loss`: 0.1319.



Métricas no conjunto de teste

95,40%

acurácia

94,41%

F1 macro

95,36%

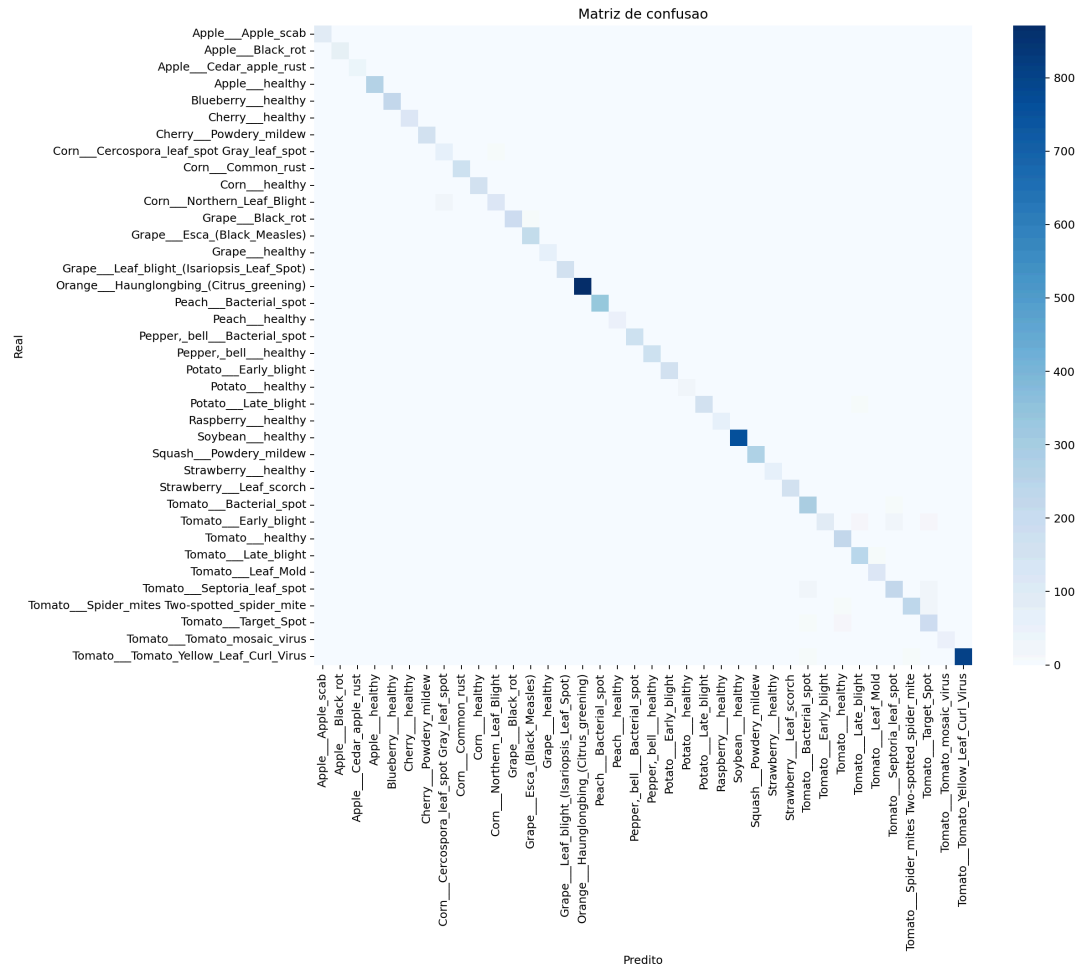
F1 ponderado

Por que acurácia?

Mede o acerto global em todas as 8.146 imagens do teste.

F1 macro vs F1 ponderado

Macro dá o mesmo peso para cada classe.
Weighted considera a quantidade de imagens por classe.



Confusões do modelo

- A matriz mostra erros recorrentes entre classes.
- Algumas confusões ocorrem entre doenças visualmente próximas.
- O relatório por classe detalha precision, recall, F1 e suporte.
- Isso orienta fine tuning e novos dados.

Como rodar uma predição

O script usa o modelo ``.keras`` e o metadata para manter a ordem correta das classes.

```
Comando: `uv run plant-predict
artifacts/models/mobilenet_full.keras imagem.jpg --
class-names
artifacts/reports/mobilenet_full_metadata.json`
```

```
pred = model.predict(image)
class_id = pred.argmax()
confidence = pred.max()
```

1

``tf.io.read_file`` e
``tf.image.decode_image(channels=3)``.

2

Resize para ``(224, 224)`` e cast para ``float32``.

3

``model.predict`` retorna 38 probabilidades.

4

``argmax`` seleciona a classe; ``max`` calcula a confiança.

Exemplos de acerto

Casos do conjunto de teste em que a previsão bateu com a classe real.

- `Blueberry\_\_healthy` → `Blueberry\_\_healthy`, confiança 100%.
- `Tomato\_\_Late\_blight` → `Tomato\_\_Late\_blight`, confiança 100%.
- `Corn\_\_Common\_rust` → `Corn\_\_Common\_rust`, confiança 100%.

Fonte:
artifacts/plots/mobilenet_full_correct_examples.png

Acertos no conjunto de teste

Real: Blueberry__healthy
Pred: Blueberry__healthy
Conf: 1.00



Real: Corn__Common_rust
Pred: Corn__Common_rust
Conf: 1.00



Real: Corn__Common_rust
Pred: Corn__Common_rust
Conf: 1.00



Real: Peach__Bacterial_spot
Pred: Peach__Bacterial_spot
Conf: 1.00



Real: Tomato__Tomato_Yellow_Leaf_Curl_Virus
Pred: Tomato__Tomato_Yellow_Leaf_Curl_Virus
Conf: 1.00



Real: Corn__Northern_Leaf_Blight
Pred: Corn__Northern_Leaf_Blight
Conf: 0.99



Real: Tomato__Late_blight
Pred: Tomato__Late_blight
Conf: 0.69



Real: Blueberry__healthy
Pred: Blueberry__healthy
Conf: 1.00



Real: Grape__Esca_(Black_Measles)
Pred: Grape__Esca_(Black_Measles)
Conf: 1.00



Real: Peach__Bacterial_spot
Pred: Peach__Bacterial_spot
Conf: 1.00



Real: Grape__Esca_(Black_Measles)
Pred: Grape__Esca_(Black_Measles)
Conf: 1.00



Real: Peach__Bacterial_spot
Pred: Peach__Bacterial_spot
Conf: 1.00



Exemplos de erro

Mesmo com boa acurácia, alguns erros aparecem com confiança alta.

- `Soybean\_\_healthy` → `Apple\_\_healthy`, confiança 99,99%.
- `Tomato\_\_Late\_blight` → `Tomato\_\_healthy`, confiança 99,98%.
- `Potato\_\_Late\_blight` → `Tomato\_\_Late\_blight`, confiança 99,98%.

Próximo passo: revisar essas classes e testar imagens fora do PlantVillage.

Erros no conjunto de teste

Real: Tomato__Spider_mites Two-spotted_spider_mite
Pred: Tomato__Bacterial_spot
Conf: 0.45



Real: Tomato__Spider_mites Two-spotted_spider_mite
Pred: Tomato__Target_Spot
Conf: 0.46



Real: Tomato__Early_blight
Pred: Tomato__Septoria_leaf_spot
Conf: 0.48



Real: Potato__Early_blight
Pred: Tomato__Late_blight
Conf: 0.58



Real: Tomato__Target_Spot
Pred: Tomato__healthy
Conf: 0.93



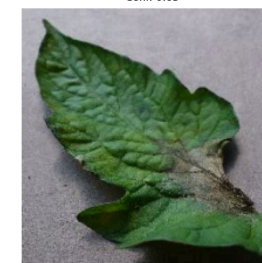
Real: Tomato__Tomato_Yellow_Leaf_Curl_Virus
Pred: Tomato__Leaf_Mold
Conf: 0.79



Real: Corn__Northern_Leaf_Blight
Pred: Corn__Cercospora_leaf_spot Gray_leaf_spot
Conf: 0.99



Real: Tomato__Late_blight
Pred: Tomato__Leaf_Mold
Conf: 0.63



Real: Tomato__healthy
Pred: Squash__Powdery_mildew
Conf: 1.00



Real: Tomato__Tomato_Yellow_Leaf_Curl_Virus
Pred: Tomato__Bacterial_spot
Conf: 0.80



Real: Tomato__Septoria_leaf_spot
Pred: Tomato__Target_Spot
Conf: 0.72



Real: Corn__Cercospora_leaf_spot Gray_leaf_spot
Pred: Corn__Northern_Leaf_Blight
Conf: 0.98



Conclusões

Modelo

MobileNetV2 com transfer learning atingiu 95,40% de acurácia no teste.

Engenharia

O projeto tem CLI, testes, metadados, modelo e artefatos versionados.

Limite

PlantVillage é uma base controlada; generalização externa exige validação adicional.

Próximos passos: fine tuning, comparação com EfficientNet e teste com imagens reais fora do PlantVillage.